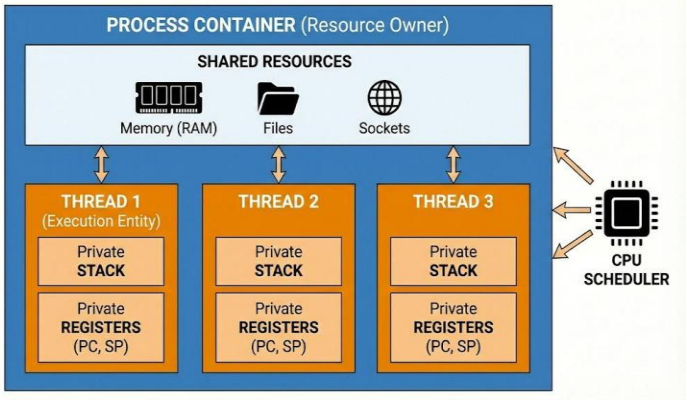
# 10. Java Threads – Introduction and Lab Experiments

**What is a process:** A process is a program that is currently executing.
Example: When you open a browser, a process is created to run the browser application.

**What is a thread:** A thread is the smallest unit of execution within a process. It performs a particular task.
Example: A browser can use different threads to load a webpage, download a file, and respond to user actions.




**Process vs. Thread**

| Process | Thread |
|---|---|
| A running program. | An execution unit within a process. |
| Has its own process address space. | Shares the process's memory with other threads. |
| Creation generally has higher overhead. | Creation generally has lower overhead. |
| Processes are isolated from one another. | Threads can communicate through shared data. |
| Example: Two independent browser applications. | Example: One browser process handling multiple tasks. |

**Why do we need threads?**
Threads help us:

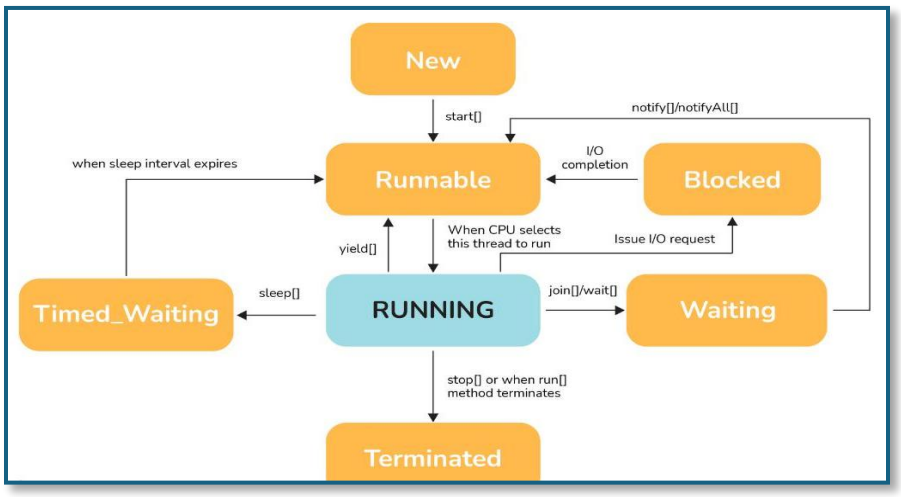
1. Perform multiple tasks concurrently.
2. Improve application responsiveness.
3. Utilize multiple CPU cores.
4. Perform background tasks without blocking the main task.
5. Share data efficiently within a process.

Real-time example: In a music player, one thread can play music while another downloads a song and another updates the user interface.

**What is multithreading:** Multithreading is the execution of multiple threads within a single process.
Threads may execute concurrently, and on a multicore CPU, they may execute in parallel.

- Concurrency: Multiple tasks make progress over overlapping periods.
- Parallelism: Multiple tasks execute simultaneously on different processing resources.

**Thread Life Cycle**



| State | Meaning |
|---|---|
| NEW | Thread object is created. |
| RUNNABLE | Thread is ready to run or running on the CPU. |
| BLOCKED | Waiting to acquire a monitor lock. |
| WAITING | Waiting indefinitely for another thread's action. |
| TIMED_WAITING | Waiting for a specified time, such as during sleep(). |
| TERMINATED | Thread execution has completed. |

**Important methods:**

**Important:** Calling start() creates a new thread of execution. Calling run() directly executes the method on the current thread.

**Java Thread Methods**

| Method | Description |
|---|---|
| start() | Starts a new thread of execution. |
| run() | Defines the task or code executed by the thread. |
| sleep(ms) | Pauses the currently executing thread temporarily for the specified time in milliseconds. |
| join() | Makes the current thread wait until another thread finishes. |
| isAlive() | Checks whether a thread has started and has not yet terminated. |
| getName()/ setName() | Gets or sets the thread's name. |
| getPriority()/ setPriority() | Gets or sets the thread's priority (1–10). |
| yield() | Hints to the scheduler that the current thread is willing to let other threads execute. |
| interrupt() | Interrupts a thread or signals its interrupted status; it does not forcibly stop the thread. |
| currentThread() | Returns the currently executing thread. |
| setDaemon()/ isDaemon() | Sets or checks whether a thread is a daemon (background) thread. setDaemon() must be called before starting the thread. |
| activeCount() | Returns an estimate of the number of active threads in the current thread's thread group. |

**Creating Threads in Java:** Java provides two common approaches.

The main difference between extending the Thread class and implementing the Runnable interface for thread creation in Java is in the way the task logic is structured and the coupling between the task and the thread. Extending the Thread class allows for tighter coupling, as the task logic is tightly coupled with the thread's execution. In contrast, implementing the Runnable interface promotes a more loosely coupled design, allowing multiple threads to share the same Runnable object and enabling the use of the Executor Framework for task execution. This separation of concerns makes it easier to manage and scale multithreaded applications.

**Method 1: Extending the Thread class**

```
class Task extends Thread {
   public void run() {
      System.out.println("Thread is running");
   }

   public static void main(String[] args) {
      Task t1 = new Task();
      t1.start();
   }
}
```

**Output:** Thread is running

**Explanation:**

1. Task extends the Thread class.
2. run() defines the work performed by the thread.
3. t1 creates a thread object.
4. start() starts a new thread, which executes run().

**Method 2: Implementing the Runnable interface**

```
class Task implements Runnable {
   public void run() {
```

```
        System.out.println("Thread is running");
    }

    public static void main(String[] args) {
        Task obj = new Task();
        Thread t1 = new Thread(obj);
        t1.start();
    }
}
```

**Output:** Thread is running

**Which method should students prefer?**

Implementing Runnable is often preferable because the class can extend another class and keep the task separate from the thread-management object.

## Experiment 10.1: Creating a Thread Using the Thread Class

**Aim:** To create a thread by extending the Thread class and overriding the run() method.
Concepts: Thread class, run() method, start() method.

A thread is a lightweight unit of execution within a process. Java allows us to create a thread by extending the Thread class and overriding its run() method. The start() method starts a new thread, which then executes the run() method.

**Algorithm**

1. Create a class MyThread that extends Thread.
2. Override the run() method.
3. Print numbers from 1 to 5 inside the run() method.
4. Create an object of MyThread in the main() method.
5. Call start() to execute the child thread.
6. Print a message from the main thread.

**Sample output**

Main Thread
Child Thread: 1
Child Thread: 2
Child Thread: 3
Child Thread: 4
Child Thread: 5

**Note: The output order may vary because the main thread and child thread execute concurrently.**

## Experiment 10.2: Creating a Thread Using the Runnable Interface

**Aim:** To create and execute a thread by implementing the Runnable interface.
**Concepts:** Runnable interface, Thread constructor, run() method, start() method.

The Runnable interface provides a run() method that defines the task to be executed. A class implements Runnable, and its object is passed to the Thread constructor. Calling start() starts a new thread that executes the task.

**Algorithm**

1. Create a class MyTask that implements Runnable.
2. Implement the run() method.
3. Print numbers from 1 to 5 inside the run() method.
4. Create an object of MyTask.
5. Create a Thread object by passing the task object to its constructor.
6. Call start() to execute the task in a separate thread.

**Sample output**

Task: 1
Task: 2
Task: 3
Task: 4
Task: 5

## Experiment 10.3: Executing Two Threads Concurrently

**Aim:** To demonstrate concurrent execution using two threads in Java.
**Concepts:** Multiple threads, Thread class, run() method, start() method, setName(), currentThread(), and thread scheduling.
**Multithreading:** Multithreading allows multiple threads to execute within the same process. Each thread performs its own task, and the JVM scheduler determines when each thread gets an opportunity to execute.
**Algorithm**

1. Create a class Task that extends the Thread class.
2. Override the run() method.
3. Use a loop to print numbers from 1 to 3.
4. Use Thread.currentThread().getName() to display the executing thread's name.
5. Create two thread objects, t1 and t2.
6. Assign the names Thread-1 and Thread-2 using setName().
7. Call start() on both threads.
8. Observe the output produced by both threads.

**Sample output**

Thread-1 - Count: 1
Thread-2 - Count: 1
Thread-1 - Count: 2
Thread-1 - Count: 3
Thread-2 - Count: 2
Thread-2 - Count: 3

**Note:** The output may vary each time the program runs. For example, all three messages from Thread-1 may appear before the messages from Thread-2. Both executions are valid.

## Experiment 10.4: Using sleep() in Threads

**Aim:** To demonstrate how the sleep() method temporarily pauses a thread during execution.
**Concepts:** Thread class, run() method, start() method, Thread.sleep(), InterruptedException.
The sleep() method in Java temporarily pauses the currently executing thread for a specified period.
**The syntax is:** Thread.sleep(milliseconds);

For example, Thread.sleep(1000) pauses the current thread for approximately one second. During this time, the thread does not continue its normal execution, allowing other eligible threads to execute.
The sleep() method may throw an InterruptedException, which must be handled or declared.

**Algorithm**

1. Create a class SleepDemo that extends the Thread class.
2. Override the run() method.
3. Use a loop to print numbers from 1 to 5.
4. Call Thread.sleep(1000) after printing each number.
5. Handle InterruptedException using a try-catch block.
6. Create an object of SleepDemo in the main() method.
7. Call start() to begin thread execution.
8. Observe the delay between successive numbers.

**Sample output**

1
2
3
4
5

The numbers appear approximately one second apart.

### Explanation

- Thread.sleep(1000): Pauses the currently executing thread for approximately 1000 milliseconds.
- try-catch: Handles the InterruptedException that may occur during sleep.
- Thread.currentThread().interrupt(): Restores the interrupted status when the exception is caught.
- return: Ends the execution of the run() method if the thread is interrupted.
- start(): Starts a new thread that executes the run() method.

**Important:** The delay is not guaranteed to be exactly one second. In this program, the delay between the first and fifth prints is approximately four seconds, plus execution and scheduling overhead. The final sleep occurs after printing 5, but it does not delay the program's visible output.

## Experiment 10.5: Executing Two Threads Sequentially Using join()

**Aim:** To execute two threads in a specific order using the join() method.
**Concepts:** Thread class, start(), join(), run(), thread coordination.

The join() method makes the currently executing thread wait until another thread completes its execution. It is used to coordinate threads and control the order in which tasks proceed.

In this experiment, Thread-1 executes first. The main thread waits for Thread-1 to finish using join(). After that, Thread-2 starts and executes its task.

**Algorithm**

1. Create a class Task that extends the Thread class.
2. Override the run() method to print numbers from 1 to 3.
3. Create two thread objects, t1 and t2.
4. Assign the names Thread-1 and Thread-2.
5. Start Thread-1 using start().
6. Call t1.join() to wait until Thread-1 completes.
7. Start Thread-2 after Thread-1 terminates.
8. Observe the output.

**Sample output**

Thread-1 - Count: 1
Thread-1 - Count: 2
Thread-1 - Count: 3
Thread-2 - Count: 1
Thread-2 - Count: 2
Thread-2 - Count: 3

**Explanation**

1. t1.start() starts Thread-1.
2. t1.join() makes the main thread wait until Thread-1 finishes.
3. After Thread-1 terminates, the main thread resumes execution.
4. t2.start() starts Thread-2.
5. Thread-2 begins execution only after Thread-1 has completed.

## Experiment 10. 6: Setting Thread Priorities

**Aim:** To demonstrate how thread priorities can be assigned and retrieved in Java.
**Concepts:** Thread class, setPriority(), getPriority(), MIN_PRIORITY, NORM_PRIORITY, MAX_PRIORITY, thread scheduling.

Thread priority indicates the relative priority assigned to a thread. Java provides priorities ranging from 1 to 10.

- Thread.MIN_PRIORITY – 1 (lowest priority)
- Thread.NORM_PRIORITY – 5 (normal priority)
- Thread.MAX_PRIORITY – 10 (highest priority)

The setPriority() method sets a thread's priority, while getPriority() returns its current priority.
Note: A higher priority does not guarantee that a thread will execute first. The actual execution order depends on the JVM and the underlying operating system's thread scheduler.

**Algorithm**

1. Create two thread objects, t1 and t2.
2. Define a task for each thread.
3. Set t1 to minimum priority using setPriority().
4. Set t2 to maximum priority using setPriority().
5. Display both priorities using getPriority().
6. Start both threads using start().
7. Observe the output.

**Sample output**

Thread 1 Priority: 1
Thread 2 Priority: 10

Task 1 running
Task 2 running

Note: The task messages may appear in either order. Thread priority does not guarantee execution order.

**Explanation**

1. new Thread(...) creates two thread objects.
2. setPriority(Thread.MIN_PRIORITY) assigns priority 1 to Thread-1.
3. setPriority(Thread.MAX_PRIORITY) assigns priority 10 to Thread-2.
4. getPriority() retrieves the assigned priorities.
5. start() starts both threads.
6. The scheduler determines when the threads execute; the higher-priority thread is not guaranteed to execute first.

## Experiment 10.7: Synchronization Using a Shared Bank Account

**Aim:** To demonstrate synchronization when multiple threads access and modify a shared resource.
**Concepts:** Shared objects, race conditions, critical sections, synchronized methods, thread coordination.

Synchronization in Java ensures that only one thread at a time can execute a synchronized method on the same object. Consider a bank account with an initial balance of ₹1,000. Two customers attempt to withdraw ₹700 and ₹500, respectively. Since both withdrawals access the same account, synchronization is required to prevent incorrect balance updates.

The synchronized keyword ensures that one withdrawal operation completes before another thread can execute the synchronized method on the same account object.

**Algorithm**

1. Create a class BankAccount with an initial balance of ₹1,000.
2. Define a synchronized method withdraw() that accepts the withdrawal amount.
3. Check whether the balance is sufficient.
4. If sufficient, deduct the amount and display the remaining balance.
5. Otherwise, display an insufficient balance message.
6. Create a single shared BankAccount object.
7. Create two threads representing two customers.
8. Start both threads and observe the output.

**Sample output — Case 1**
Customer-1 withdrew 700, Balance: 300
Customer-2: Insufficient balance
**Sample output — Case 2**
Customer-2 withdrew 500, Balance: 500
Customer-1: Insufficient balance
Note: Both outputs are valid because either customer may acquire the account's monitor first. However, only one withdrawal succeeds because the total requested amount exceeds the initial balance.

**Explanation**

1. BankAccount account = new BankAccount() creates one shared account.
2. Both threads access the same account object.
3. synchronized allows only one thread at a time to execute withdraw() on that object.
4. The first customer withdraws either ₹700 or ₹500.
5. The remaining balance is insufficient for the second withdrawal.
6. Therefore, the balance cannot become negative in this example.